

熊猫API框架

PandaAPI Framework

PHP 7.4+ | PandaDB | inhere/sroute

中文用户手册 v1.0

目录

1. 简介
 2. 核心特性
 3. 快速开始
 4. PandaDB 数据库操作
 5. 缓存系统
 6. 路由系统
 7. 请求与响应
 8. 中间件
 9. 验证器
 10. 性能优化
 11. 配置参考
- 附录 A 目录结构
- 附录 B 许可证

1. 简介

熊猫API框架 (PandaAPI) 是一个基于 PHP 7.4+ 的高性能 API 框架，融合了 ThinkPHP 的简洁链式调用语法和 PandaDB 的快捷操作方法，同时集成 inhere/sroute 的极速路由匹配。框架默认关闭 PHP Session，实现无状态设计，最大化运行性能。

1.1 主要特点

PandaDB 数据库类 - 融合 ThinkPHP 链式调用和 PandaDB 快捷方法

ThinkPHP 风格 - 熟悉的 `Db::table()->where()->select()` 语法

PandaDB 兼容 - 支持 `select/get/insert/update/delete` 等快捷方法

多缓存后端 - File / Memcache / Redis / OpenResty

极速路由匹配 - $O(1)$ 时间复杂度，不受路由数量影响

无状态设计 - 关闭 PHP Session，最大化性能

PDO 预处理 - 防止 SQL 注入

查询性能统计 - 识别慢查询

2. 核心特性

2.1 PandaDB 数据库类

框架提供 PandaDB 类，融合两种流行的数据库操作方式：

ThinkPHP 风格链式调用：

```
PandaDB::table('user')->where('status', 1)->select(); PandaDB::table('user')->where('id', 1)->find(); PandaDB::table('user')->insert(['name' => 'test']);
PandaDB::table('user')->where('id', 1)->update(['name' => 'new']);
PandaDB::table('user')->where('id', 1)->delete();
```

PandaDB 风格快捷方法：

```
PandaDB::select('user', '*', ['status' => 1]); PandaDB::get('user', '*', ['id' => 1]);
PandaDB::insert('user', ['name' => 'test']); PandaDB::update('user', ['name' => 'new'], ['id' => 1]);
PandaDB::delete('user', ['id' => 1]);
```

2.2 性能优化措施

特性	说明
PDO 预处理	原生 PDO 预处理，防止 SQL 注入
查询缓存	支持多种缓存后端自动缓存
查询构建器	链式调用，延迟执行
性能统计	记录查询次数、执行时间，识别慢查询

2.3 路由匹配优势

与 inhere/sroute 融合，具备以下优势：

- 极速路由匹配 - O(1) 时间复杂度，不受路由数量影响
- 支持路由组 - 便于 API 版本管理
- 中间件支持 - 灵活的请求处理管道
- 命名路由 - 便于生成 URL
- 参数匹配 - 支持动态路径参数

3. 快速开始

3.1 安装

```
composer require pandaapi/framework
```

3.2 配置数据库

```
use PandaAPI\Database\PandaDB; PandaDB::config([ 'driver' => 'mysql', 'host' => '127.0.0.1',  
'port' => 3306, 'database' => 'pandaapi', 'username' => 'root', 'password' => '', ]);
```

3.3 配置缓存

```
use PandaAPI\Cache\Cache; Cache::config([ 'driver' => 'redis', 'redis' => [ 'host' =>  
'127.0.0.1', 'port' => 6379, ], ]);
```

3.4 定义路由

```
use PandaAPI\Route\Route; use PandaAPI\Core\Response; Route::get('/users', function() { return  
Response::json(PandaDB::table('user')->select()); }); Route::get('/user/{id}', function($id) {  
return Response::json( PandaDB::table('user')->where('id', $id)->find() ); });
```

3.5 运行应用

```
use PandaAPI\Core\App; $app = new App(); $app->run();
```

4. PandaDB 数据库操作

4.1 ThinkPHP 风格链式方法

方法	说明
table(\$table)	设置表名
name(\$name)	设置表名（自动添加前缀）
field(\$fields)	指定查询字段（字符串或数组）
where(\$field, \$value)	WHERE 等值条件
whereOr(\$field, \$value)	OR WHERE 条件
whereIn(\$field, \$values)	WHERE IN 条件
whereNotIn(\$field, \$values)	WHERE NOT IN 条件
whereBetween(\$field, \$min, \$max)	WHERE BETWEEN 条件
whereLike(\$field, \$value)	WHERE LIKE 模糊匹配
whereNull(\$field)	WHERE IS NULL
whereNotNull(\$field)	WHERE IS NOT NULL
join(\$table, \$on, \$type)	JOIN 关联查询
leftJoin(\$table, \$on)	LEFT JOIN
rightJoin(\$table, \$on)	RIGHT JOIN
order(\$field, \$dir)	排序（ASC / DESC）
group(\$field)	分组
having(\$cond)	HAVING 条件
limit(\$n, \$offset)	限制条数
page(\$page, \$perPage)	分页查询

4.2 PandaDB 风格快捷方法

方法	说明
select(\$table, \$columns, \$where)	查询多条记录
get(\$table, \$columns, \$where)	查询单条记录
insert(\$table, \$data)	插入数据，返回自增 ID
insertAll(\$table, \$dataSet)	批量插入
update(\$table, \$data, \$where)	更新数据，返回影响行数
delete(\$table, \$where)	删除数据，返回影响行数

count(\$table, \$where)	统计记录数量
sum(\$table, \$column, \$where)	求和
avg(\$table, \$column, \$where)	平均值
max(\$table, \$column, \$where)	最大值
min(\$table, \$column, \$where)	最小值

4.3 查询示例

```
// ThinkPHP 00 PandaDB::table('users')->select(); // 0000
PandaDB::table('users')->where('status', 1)->select(); // 0000 PandaDB::table('users')
->field('id, name, email') ->where('status', 1) ->order('created_at', 'desc') ->limit(10)
->select(); // PandaDB 00 PandaDB::select('users', '*', ['status' => 1]); // 0000 $result =
PandaDB::table('users')->paginate(15, $page); // $result['data'] - 0000 // $result['total'] -
0000 // $result['current_page'] - 0000 // $result['last_page'] - 0000 // $result['per_page'] -
0000
```

4.4 写入示例

```
// ThinkPHP 00 - 0000 $id = PandaDB::table('users')->insert([ 'name' => '00', 'email' =>
'zhangsan@test.com', 'password' => password_hash('123456', PASSWORD_DEFAULT), ]); // PandaDB 00 -
00 $id = PandaDB::insert('users', [ 'name' => '00', 'email' => 'zhangsan@test.com', ]); // 0000
PandaDB::table('users')->insertAll([ ['name' => 'A', 'email' => 'a@test.com'], ['name' => 'B',
'email' => 'b@test.com'], ]); // ThinkPHP 00 - 00 PandaDB::table('users')->where('id',
1)->update(['name' => '000']); // PandaDB 00 - 00 PandaDB::update('users', ['name' => '000'],
['id' => 1]); // 00 PandaDB::table('users')->where('id', 1)->delete(); PandaDB::delete('users',
['id' => 1]);
```

4.5 聚合查询

```
// ThinkPHP 00 PandaDB::table('users')->count(); PandaDB::table('orders')->sum('amount');
PandaDB::table('users')->avg('score'); PandaDB::table('products')->max('price');
PandaDB::table('products')->min('price'); // PandaDB 00 PandaDB::count('users');
PandaDB::sum('orders', 'amount'); PandaDB::avg('users', 'score'); PandaDB::max('products',
'price'); PandaDB::min('products', 'price');
```

4.6 事务操作

```
// 00000000 PandaDB::transaction(function() { PandaDB::table('users')->insert([...]);
PandaDB::table('logs')->insert([...]); }); // 00000000 PandaDB::beginTransaction(); try {
PandaDB::table('users')->insert([...]); PandaDB::commit(); } catch (Exception $e) {
PandaDB::rollback(); }
```

4.7 原生 SQL

```
// 00 $rows = PandaDB::query('SELECT * FROM users WHERE status = ?', [1]); // 00 $affected =
PandaDB::execute('DELETE FROM logs WHERE created_at < ?', ['2024-01-01']); // 0000 $row =
PandaDB::getOne('SELECT * FROM users WHERE id = ?', [1]); // 00000 $count =
PandaDB::getValue('SELECT COUNT(*) FROM users'); // 00000 $names = PandaDB::getColumn('SELECT
name FROM users');
```

5. 缓存系统

5.1 支持的缓存驱动

驱动	配置项	说明
file	path	文件缓存，无需额外服务
redis	host, port, password	高性能内存缓存
memcache	host, port	分布式内存缓存
openresty	host, port	OpenResty 共享内存缓存

5.2 配置示例

```
// Redis
Cache::config([ 'driver' => 'redis', 'redis' => [ 'host' => '127.0.0.1', 'port' => 6379, 'password' => null, 'database' => 0, 'prefix' => 'pandaapi:', ], ], ); //
Cache::config([ 'driver' => 'file', 'file' => [ 'path' => '/tmp/cache', ], ], );
```

5.3 基本操作

```
//
Cache::set('key', 'value', 300); //
Cache::get('key', 'default'); //
Cache::has('key'); //
Cache::delete('key'); //
Cache::forget('key'); //
Cache::clear(); //
$users = Cache::remember('user_list', function() { return PandaDB::table('users')->select(); }, 300); //
Cache::pull('key'); //
Cache::increment('counter', 1); Cache::decrement('counter', 1); //
Cache::setMultiple(['k1'=>'v1', 'k2'=>'v2'], 300); Cache::getMultiple(['k1', 'k2']);
Cache::deleteMultiple(['k1', 'k2']);
```

5.4 标签缓存

```
Cache::tag('users')->set('list', $data); Cache::tag('users')->set('count', $count);
Cache::tag('users')->flush();
```

5.5 缓存锁

```
if (Cache::lock('resource:1', 10)) {
    Cache::unlock('resource:1');
}
```


6. 路由系统

6.1 HTTP 方法路由

```
Route::get($path, $handler); // GET Route::post($path, $handler); // POST Route::put($path, $handler); // PUT Route::patch($path, $handler); // PATCH Route::delete($path, $handler); // DELETE Route::options($path, $handler); // OPTIONS Route::any($path, $handler); // 任意 Route::match(['GET', 'POST'], $path, $handler); // 匹配
```

6.2 路由组

```
// 路由组 Route::group(['prefix' => '/api/v1'], function() { Route::get('/users', 'UserController@index'); Route::get('/users/{id}', 'UserController@show'); }); // 中间件 Route::group(['middleware' => ['Auth', 'Cors']], function() { Route::get('/profile', 'ProfileController@show'); }); // 子路由 Route::group(['prefix' => '/api'], function() { Route::group(['prefix' => '/v1'], function() { Route::get('/users', 'UserController@index'); }); });
```

6.3 参数约束

```
// 数字 Route::get('/user/{id}', 'Controller@show') ->whereNumber('id'); // 字符串 Route::get('/post/{slug}', 'Controller@show') ->whereAlpha('slug'); // 字母 Route::get('/tag/{name}', 'Controller@show') ->whereAlphaNum('name'); // UUID Route::get('/order/{uuid}', 'Controller@show') ->whereUuid('uuid'); // 正则 Route::get('/file/{path}', 'Controller@show') ->where('path', '[a-z0-9/]+');
```

6.4 资源路由

```
Route::resource('users', 'UserController'); // 资源路由 // GET /users index (00) // GET /users/{id} show (00) // POST /users store (00) // PUT /users/{id} update (00) // DELETE /users/{id} destroy (00)
```

6.5 其他路由功能

```
// 路由名称 Route::get('/profile', 'Controller@show')->name('profile'); $url = Route::route('profile'); // 重定向 Route::redirect('/old', '/new', 301); // 404 路由 Route::set404(function($path) { return Response::error('404', 404); });
```

7. 请求与响应

7.1 Request 请求方法

方法	说明
method()	获取 HTTP 方法 (GET / POST 等)
uri()	获取请求路径
url()	获取完整 URL
all()	获取所有参数 (GET + POST)
query(\$key)	获取 GET 参数
post(\$key)	获取 POST 参数
header(\$key)	获取请求头
json()	获取 JSON 请求体 (自动解析)
file(\$key)	获取上传文件
cookie(\$key)	获取 Cookie
ip()	获取客户端 IP
user()	获取当前认证用户
isAjax()	是否 AJAX 请求
isJson()	是否 JSON 请求

7.2 Response 响应方法

```
// JSON 返回 Response::json(['data' => $value]); Response::json($data, 201); // 成功 // 失败 /
// 成功 Response::success(['user' => $user]); Response::error('失败', 500); // 失败
Response::paginate($data, $total, $page, $perPage); // 分页 Response::text('文本');
Response::html('<h1>HTML</h1>'); Response::redirect('/new-url', 301); Response::noContent(); //
204 // 成功 Response::json($data) ->status(201) ->header(['X-Custom' => 'value']) ->send();
```

8. 中间件

8.1 内置中间件

中间件	类名	说明
CORS	CorsMiddleware	跨域资源共享处理
Auth	AuthMiddleware	Token 用户认证
Throttle	ThrottleMiddleware	请求频率限流
Log	LogMiddleware	请求日志记录
Validate	ValidateMiddleware	请求数据验证

8.2 使用方式

```
// 中间件注册
$app->middleware(['Cors', 'Log']); // 路由组应用中间件
Route::group(['middleware' => ['Auth']], function() {
    Route::get('/profile', 'ProfileController@show');
}); // 特定路由应用中间件
Route::get('/admin', 'AdminController@index')->middleware(['Auth', 'Throttle']);
```

8.3 自定义中间件

```
class MyMiddleware {
    public function handle($request, callable $next) {
        // 处理逻辑
        $response = $next($request);
        // 返回响应
        return $response;
    }
}

// 注册中间件
Middleware::register('my', MyMiddleware::class);
```

9. 验证器

9.1 内置验证规则

参数	说明
-	必填
-	合法邮箱
n	最小长度或最小值
n	最大长度或最大值
-	数字
-	整数
-	字符串
-	数组
v1,v2,...	值在列表中
v1,v2,...	值不在列表中
min,max	在范围内
pattern	匹配正则
-	合法 URL
-	合法 IP
-	仅字母
-	字母和数字
format	合法日期（默认 Y-m-d）
-	与字段_confirmation 匹配

9.2 使用示例

```
$validator = new Validator($data, [ 'name' => 'required|string|min:2|max:50', 'email' =>
'required|email', 'password' => 'required|string|min:6|confirmed', 'age' =>
'numeric|between:18,100', ]); if ($validator->fails()) { return Response::json([ 'error' =>
'!!!!', 'messages' => $validator->errors(), ], 422); } // 验证失败 $validated =
$validator->validated(); // 验证成功 $firstError = $validator->firstError();
```

10. 性能优化

10.1 查询缓存

```
// 通过 Cache::remember 缓存查询结果 $users = Cache::remember('active_users', function() { return  
PandaDB::table('users') ->where('status', 1) ->order('created_at', 'desc') ->limit(50)  
->select(); }, 300); // 最多缓存 5 秒
```

10.2 性能统计

```
// 获取查询日志 $queries = PandaDB::getQueryLog(); // 获取查询次数 PandaDB::getQueryCount(); // 获取  
PandaDB::getTotalQueryTime(); // 获取最后一次查询时间 PandaDB::getLastQueryTime(); // 获取慢查询  
PandaDB::getSlowQueries(); // 设置慢查询阈值 PandaDB::setSlowThreshold(100);
```

10.3 无状态设计

框架默认关闭 PHP Session，通过 Token 机制实现认证，避免 Session 读写带来的 I/O 开销，最大化并发性能。入口文件中已包含以下处理：

```
// 关闭 Session if (session_status() == PHP_SESSION_ACTIVE) { session_write_close(); }  
// 设置 Session 自动启动为 0 ini_set('session.auto_start', '0');
```

11. 配置参考

11.1 完整配置示例

```
return [ 'env' => 'development', 'debug' => true, 'timezone' => 'Asia/Shanghai', // 配置数据库
'database' => [ 'driver' => 'mysql', 'host' => '127.0.0.1', 'port' => 3306, 'database' =>
'pandaapi', 'username' => 'root', 'password' => '', 'charset' => 'utf8mb4', 'prefix' => '', ], // 配置缓存
'cache' => [ 'driver' => 'redis', 'redis' => [ 'host' => '127.0.0.1', 'port' => 6379,
'password' => null, 'database' => 0, 'prefix' => 'pandaapi:', ], 'default_ttl' => 3600, ],
'middleware' => ['Cors', 'Log'], 'throttle' => [ 'max_attempts' => 60, 'decay_minutes' => 1, ],
'cors' => [ 'origin' => '*', 'methods' => 'GET, POST, PUT, DELETE, OPTIONS', 'headers' =>
'Content-Type, Authorization', ], ];
```

附录 A 目录结构

```
pandaapi/ +-- src/ | +-- Core/ #  (App, Request, Response) | +-- Database/ # PandaDB  |
+-- Cache/ #  (Cache, Drivers, QueryCache) | +-- Route/ #  (Route, RouteItem, Middleware) |
+-- Middleware/ #  | +-- Validation/ #  | +-- Exception/ #  +-- demo/ #  | +--
index.php #  | +-- Controllers.php #  | +-- config.php #  | +-- init_database.php#
| +-- api_test.php # API  +-- app/Controller/ #  +-- config/ #  +-- routes/ #
+-- tests/ #  +-- public/ # Web  +-- composer.json
```

附录 B 许可证

MIT License - 可自由使用、修改和分发。

(c) 2024 熊猫API框架 (PandaAPI). 保留所有权利。